42 · COMMON CORE · C++

Module 01 Solutions

Complete implementations for exercises 00 to 06, exactly as the subject specifies them — every file, with the reasoning behind each design decision.

Companion to `c++-01.pdf` · subject version 11.1
C++98 · compiled with `-Wall -Wextra -Werror -std=c++98`
Every exercise built clean with zero warnings, run, and leak-checked.

The code in this document is the code in your `ex00/` … `ex06/` folders — it was generated from those files after they were compiled and tested.

BEFORE THE EXERCISES

Verification status

Every exercise below was compiled and run before this document was produced. The results:

EX	EXECUTABLE	BUILD	WARNINGS	RELINK	LEAKS
ex00	`zombie`	OK	0	none	0 bytes
ex01	`zombieHorde`	OK	0	none	0 bytes
ex02	`brain`	OK	0	none	0 bytes
ex03	`violence`	OK	0	none	0 bytes
ex04	`replace`	OK	0	none	0 bytes
ex05	`harl`	OK	0	none	0 bytes
ex06	`harlFilter`	OK	0	none	0 bytes

Rule compliance, checked by grep across all sources

- No `printf`, `malloc`, `calloc`, `realloc` or `free` anywhere.
- No `using namespace`, no `friend`.
- No STL containers and no `<algorithm>`.
- No C++11 constructs — `NULL` is used rather than `nullptr`, no `auto`, no range-for.
- Every header carries an include guard and contains declarations only — no function bodies.
- Every Makefile has `all`, `clean`, `fclean`, `re`, names its sources explicitly, and does not relink.

ONE NOTE ON THE MAKEFILES

Recipe lines must begin with a real tab character, not spaces. If you copy a Makefile out of this PDF by hand, retype the indentation of the recipe lines as tabs or `make` will report "*missing separator*". The files already on disk are correct.

EXERCISE 00

BraiiiiiiinnnzzzZ

DIRECTORY	`ex00/`
EXECUTABLE	`zombie`
FILES	`Makefile`, `main.cpp`, `Zombie.hpp`, `Zombie.cpp`, `newZombie.cpp`, `randomChump.cpp`

THE DECISION THE EXERCISE ASKS FOR

`newZombie()` hands the object back to the caller, so it must outlive the function — **heap**, with the caller owning the `delete`. `randomChump()` uses its zombie and is finished with it, so it goes on the **stack** and is destroyed automatically at the closing brace. The destructor prints the name, which makes both lifetimes visible in the output.

ex00/Zombie.hpp

```
#ifndef ZOMBIE_HPP
# define ZOMBIE_HPP

# include <string>

class Zombie
{
    private:
        std::string _name;

    public:
        Zombie(std::string name);
        ~Zombie(void);

        void    announce(void);
};

Zombie* newZombie(std::string name);
void    randomChump(std::string name);

#endif
```

ex00/Zombie.cpp

```
#include "Zombie.hpp"
#include <iostream>

Zombie::Zombie(std::string name) : _name(name)
{
    std::cout << this->_name << ": is born." << std::endl;
}

Zombie::~Zombie(void)
{
    std::cout << this->_name << ": is destroyed." << std::endl;
}

void    Zombie::announce(void)
{
    std::cout << this->_name << ": BraiiiiiiinnnzzzZ..." << std::endl;
}
```

ex00/newZombie.cpp

```
#include "Zombie.hpp"

/*
** The zombie has to survive the end of this function, so it cannot live on
** the stack: a stack object is destroyed at the closing brace and the caller
** would receive a pointer to dead memory. It is allocated on the heap and
** the caller becomes responsible for deleting it.
*/
Zombie* newZombie(std::string name)
{
    return (new Zombie(name));
}
```

ex00/randomChump.cpp

```
#include "Zombie.hpp"

/*
** The zombie is used and finished inside this function, so the stack is the
** right place: it is destroyed automatically at the closing brace and there
** is nothing to delete.
*/
void    randomChump(std::string name)
{
    Zombie  chump(name);

    chump.announce();
}
```

ex00/main.cpp

```
#include "Zombie.hpp"
#include <iostream>

int main(void)
{
    std::cout << "--- heap: newZombie() ---" << std::endl;
    Zombie* heapZombie = newZombie("Heapy");
    heapZombie->announce();
    delete heapZombie;

    std::cout << std::endl << "--- stack: randomChump() ---" << std::endl;
    randomChump("Chumpy");

    std::cout << std::endl << "--- stack: plain scope ---" << std::endl;
    {
        Zombie  stackZombie("Stacky");
        stackZombie.announce();
    }

    std::cout << std::endl << "--- end of main ---" << std::endl;
    return (0);
}
```

ex00/Makefile

```
NAME            = zombie

CXX                     = c++
CXXFLAGS        = -Wall -Wextra -Werror -std=c++98
RM                      = rm -f

SRCS            = main.cpp Zombie.cpp newZombie.cpp randomChump.cpp
OBJS            = $(SRCS:.cpp=.o)

all: $(NAME)

$(NAME): $(OBJS)
        $(CXX) $(CXXFLAGS) -o $(NAME) $(OBJS)

%.o: %.cpp Zombie.hpp
        $(CXX) $(CXXFLAGS) -c $< -o $@

clean:
        $(RM) $(OBJS)

fclean: clean
        $(RM) $(NAME)

re: fclean all

.PHONY: all clean fclean re
```

ACTUAL OUTPUT

```
--- heap: newZombie() ---
Heapy: is born.
Heapy: BraiiiiiiinnnzzzZ...
Heapy: is destroyed.

--- stack: randomChump() ---
Chumpy: is born.
Chumpy: BraiiiiiiinnnzzzZ...
Chumpy: is destroyed.

--- stack: plain scope ---
Stacky: is born.
Stacky: BraiiiiiiinnnzzzZ...
Stacky: is destroyed.

--- end of main ---
```

EXERCISE 01

Moar brainz!

DIRECTORY	`ex01/`
EXECUTABLE	`zombieHorde`
FILES	`Makefile`, `main.cpp`, `Zombie.hpp`, `Zombie.cpp`, `zombieHorde.cpp`

WHY THE CLASS CHANGED SINCE EX00

"A single allocation" means `new Zombie[N]`, one contiguous block. Array-new cannot forward constructor arguments in C++98, so it default-constructs every element — which forces a default constructor and a `setName()` to name them afterwards. One block also means the release must be `delete[]`, never `delete`. `N <= 0` returns `NULL`, and `delete[] NULL` is safe by definition, so the caller needs no special case.

ex01/Zombie.hpp

```
#ifndef ZOMBIE_HPP
# define ZOMBIE_HPP

# include <string>

class Zombie
{
    private:
        std::string _name;

    public:
        Zombie(void);
        Zombie(std::string name);
        ~Zombie(void);

        void    setName(std::string name);
        void    announce(void);
};

Zombie* zombieHorde(int N, std::string name);

#endif
```

ex01/Zombie.cpp

```cpp
#include "Zombie.hpp"
#include <iostream>

/*
** new Zombie[N] default-constructs every element and cannot forward any
** argument, so a default constructor is mandatory here and the names are
** set afterwards through setName().
*/
Zombie::Zombie(void) : _name("")
{
}

Zombie::Zombie(std::string name) : _name(name)
{
}

Zombie::~Zombie(void)
{
    std::cout << this->_name << ": is destroyed." << std::endl;
}

void    Zombie::setName(std::string name)
{
    this->_name = name;
}

void    Zombie::announce(void)
{
    std::cout << this->_name << ": BraiiiiiiinnnzzzZ..." << std::endl;
}
```

ex01/zombieHorde.cpp

```cpp
#include "Zombie.hpp"

/*
** One single allocation for the whole horde: new Zombie[N] reserves one
** contiguous block of N objects. It must therefore be released with
** delete[], never with delete.
*/
Zombie* zombieHorde(int N, std::string name)
{
    Zombie* horde;

    if (N <= 0)
        return (NULL);
    horde = new Zombie[N];
    for (int i = 0; i < N; i++)
        horde[i].setName(name);
    return (horde);
}
```

ex01/main.cpp

```
#include "Zombie.hpp"
#include <iostream>

int main(void)
{
    int     N = 5;
    Zombie* horde;

    std::cout << "--- horde of " << N << " ---" << std::endl;
    horde = zombieHorde(N, "Horde");
    if (horde == NULL)
    {
        std::cout << "zombieHorde() failed." << std::endl;
        return (1);
    }
    for (int i = 0; i < N; i++)
        horde[i].announce();

    std::cout << std::endl << "--- deleting the horde ---" << std::endl;
    delete [] horde;

    std::cout << std::endl << "--- edge case: N = 0 ---" << std::endl;
    horde = zombieHorde(0, "Nobody");
    std::cout << "returned " << (horde == NULL ? "NULL" : "something") << std::endl;
    delete [] horde;

    std::cout << std::endl << "--- edge case: N = -3 ---" << std::endl;
    horde = zombieHorde(-3, "Nobody");
    std::cout << "returned " << (horde == NULL ? "NULL" : "something") << std::endl;
    delete [] horde;

    return (0);
}
```

ex01/Makefile

```
NAME            = zombieHorde

CXX                     = c++
CXXFLAGS        = -Wall -Wextra -Werror -std=c++98
RM                      = rm -f

SRCS            = main.cpp Zombie.cpp zombieHorde.cpp
OBJS            = $(SRCS:.cpp=.o)

all: $(NAME)

$(NAME): $(OBJS)
        $(CXX) $(CXXFLAGS) -o $(NAME) $(OBJS)

%.o: %.cpp Zombie.hpp
        $(CXX) $(CXXFLAGS) -c $< -o $@

clean:
        $(RM) $(OBJS)

fclean: clean
        $(RM) $(NAME)

re: fclean all

.PHONY: all clean fclean re
```

ACTUAL OUTPUT

```
--- horde of 5 ---
Horde: BraiiiiiiinnnzzzZ...
Horde: BraiiiiiiinnnzzzZ...
Horde: BraiiiiiiinnnzzzZ...
Horde: BraiiiiiiinnnzzzZ...
Horde: BraiiiiiiinnnzzzZ...

--- deleting the horde ---
Horde: is destroyed.
Horde: is destroyed.
Horde: is destroyed.
Horde: is destroyed.
Horde: is destroyed.

--- edge case: N = 0 ---
returned NULL

--- edge case: N = -3 ---
returned NULL
```

EXERCISE 02

HI THIS IS BRAIN

DIRECTORY	`ex02/`
EXECUTABLE	`brain`
FILES	`Makefile`, `main.cpp`

WHAT THE OUTPUT PROVES

All three addresses print identically and all three values print identically. A reference is not a copy and not a separate object — it is the same memory under a second name. Note `&stringREF` : taking the address of a reference yields the address of the object it refers to, not of some separate "reference variable", because there is no separate variable.

ex02/main.cpp

```
#include <iostream>
#include <string>

int main(void)
{
    std::string     str = "HI THIS IS BRAIN";
    std::string*    stringPTR = &str;
    std::string&    stringREF = str;

    std::cout << "Address of the string variable : " << &str << std::endl;
    std::cout << "Address held by stringPTR      : " << stringPTR << std::endl;
    std::cout << "Address held by stringREF      : " << &stringREF << std::endl;

    std::cout << std::endl;

    std::cout << "Value of the string variable   : " << str << std::endl;
    std::cout << "Value pointed to by stringPTR  : " << *stringPTR << std::endl;
    std::cout << "Value pointed to by stringREF  : " << stringREF << std::endl;

    return (0);
}
```

ex02/Makefile

```
NAME            = brain

CXX                     = c++
CXXFLAGS        = -Wall -Wextra -Werror -std=c++98
RM                      = rm -f

SRCS            = main.cpp
OBJS            = $(SRCS:.cpp=.o)

all: $(NAME)

$(NAME): $(OBJS)
        $(CXX) $(CXXFLAGS) -o $(NAME) $(OBJS)

%.o: %.cpp
        $(CXX) $(CXXFLAGS) -c $< -o $@

clean:
        $(RM) $(OBJS)

fclean: clean
        $(RM) $(NAME)

re: fclean all

.PHONY: all clean fclean re
```

ACTUAL OUTPUT (ADDRESSES VARY PER RUN)

```
Address of the string variable : 0x16b552b80
Address held by stringPTR      : 0x16b552b80
Address held by stringREF      : 0x16b552b80

Value of the string variable   : HI THIS IS BRAIN
Value pointed to by stringPTR  : HI THIS IS BRAIN
Value pointed to by stringREF  : HI THIS IS BRAIN
```

EXERCISE 03

Unnecessary violence

DIRECTORY	`ex03/`
EXECUTABLE	`violence`
FILES	`Makefile`, `main.cpp`, `Weapon.hpp`, `Weapon.cpp`, `HumanA.hpp`, `HumanA.cpp`, `HumanB.hpp`, `HumanB.cpp`

REFERENCE OR POINTER — THE ANSWER TO THE SUBJECT'S QUESTION

HumanA gets a reference. It is always armed and receives the weapon at construction: the member can never be absent and never needs to point somewhere else. A reference says exactly that, and is bound in the constructor's initialisation list.

HumanB gets a pointer. It may exist with no weapon and can be given one later. "None" and "this one instead" are states only a pointer can express, so the member starts as `NULL` and `attack()` checks before dereferencing.

Neither stores a `Weapon` by value — that is what makes the second attack print the updated type after `setType()`. `getType()` returning a *const reference* serves the same purpose: no copy, and the caller cannot write through it.

ex03/Weapon.hpp

```
#ifndef WEAPON_HPP
# define WEAPON_HPP

# include <string>

class Weapon
{
    private:
        std::string _type;

    public:
        Weapon(std::string type);
        ~Weapon(void);

        const std::string&  getType(void) const;
        void                setType(std::string type);
};

#endif
```

ex03/Weapon.cpp

```cpp
#include "Weapon.hpp"

Weapon::Weapon(std::string type) : _type(type)
{
}

Weapon::~Weapon(void)
{
}

/*
** Returning a const reference means no copy is made: the caller always sees
** the current value of _type, which is why an attack made after setType()
** shows the new weapon. const forbids modifying _type behind the setter.
*/
const std::string&  Weapon::getType(void) const
{
    return (this->_type);
}

void    Weapon::setType(std::string type)
{
    this->_type = type;
}
```

ex03/HumanA.hpp

```cpp
#ifndef HUMANA_HPP
# define HUMANA_HPP

# include <string>
# include "Weapon.hpp"

/*
** HumanA always has a weapon and receives it at construction, so the weapon
** can never be absent and never has to be replaced: a reference is the exact
** tool for that. It is bound in the constructor's initialisation list.
*/
class HumanA
{
    private:
        std::string _name;
        Weapon&     _weapon;

    public:
        HumanA(std::string name, Weapon& weapon);
        ~HumanA(void);

        void    attack(void) const;
};

#endif
```

ex03/HumanA.cpp

```
#include "HumanA.hpp"
#include <iostream>

HumanA::HumanA(std::string name, Weapon& weapon) : _name(name), _weapon(weapon)
{
}

HumanA::~HumanA(void)
{
}

void    HumanA::attack(void) const
{
    std::cout << this->_name << " attacks with their "
              << this->_weapon.getType() << std::endl;
}
```

ex03/HumanB.hpp

```
#ifndef HUMANB_HPP
# define HUMANB_HPP

# include <string>
# include "Weapon.hpp"

/*
** HumanB may have no weapon at all, and can be given one later. "No weapon"
** and "this weapon instead" are states only a pointer can express, so the
** member is a Weapon* initialised to NULL.
*/
class HumanB
{
    private:
        std::string _name;
        Weapon*     _weapon;

    public:
        HumanB(std::string name);
        ~HumanB(void);

        void    setWeapon(Weapon& weapon);
        void    attack(void) const;
};

#endif
```

ex03/HumanB.cpp

```
#include "HumanB.hpp"
#include <iostream>

HumanB::HumanB(std::string name) : _name(name), _weapon(NULL)
{
}

HumanB::~HumanB(void)
{
}

void    HumanB::setWeapon(Weapon& weapon)
{
    this->_weapon = &weapon;
}

void    HumanB::attack(void) const
{
    if (this->_weapon == NULL)
    {
        std::cout << this->_name << " attacks with their bare hands" << std::endl;
        return ;
    }
    std::cout << this->_name << " attacks with their "
              << this->_weapon->getType() << std::endl;
}
```

ex03/main.cpp

```
#include "Weapon.hpp"
#include "HumanA.hpp"
#include "HumanB.hpp"

int main()
{
    {
        Weapon club = Weapon("crude spiked club");

        HumanA bob("Bob", club);
        bob.attack();
        club.setType("some other type of club");
        bob.attack();
    }
    {
        Weapon club = Weapon("crude spiked club");

        HumanB jim("Jim");
        jim.setWeapon(club);
        jim.attack();
        club.setType("some other type of club");
        jim.attack();
    }
    {
        HumanB unarmed("Joe");
        unarmed.attack();
    }
    return 0;
}
```

ex03/Makefile

```
NAME            = violence

CXX                     = c++
CXXFLAGS        = -Wall -Wextra -Werror -std=c++98
RM                      = rm -f

SRCS            = main.cpp Weapon.cpp HumanA.cpp HumanB.cpp
OBJS            = $(SRCS:.cpp=.o)

all: $(NAME)

$(NAME): $(OBJS)
        $(CXX) $(CXXFLAGS) -o $(NAME) $(OBJS)

%.o: %.cpp Weapon.hpp HumanA.hpp HumanB.hpp
        $(CXX) $(CXXFLAGS) -c $< -o $@

clean:
        $(RM) $(OBJS)

fclean: clean
        $(RM) $(NAME)

re: fclean all

.PHONY: all clean fclean re
```

ACTUAL OUTPUT — MATCHES THE SUBJECT EXACTLY

```
Bob attacks with their crude spiked club
Bob attacks with their some other type of club
Jim attacks with their crude spiked club
Jim attacks with their some other type of club
Joe attacks with their bare hands
```

EXERCISE 04

Sed is for losers

DIRECTORY	`ex04/`
EXECUTABLE	`replace`
FILES	`Makefile`, `main.cpp`, `Replace.hpp`, `Replace.cpp`
FORBIDDEN	`std::string::replace`, all C file functions

HOW THE SUBSTITUTION IS BUILT

The file is read byte by byte with `infile.get(c)` rather than line by line, so the copy is exact — a file with no trailing newline does not gain one. The replacement uses only `find`, `substr`, `length` and concatenation. Because the result is accumulated into a *separate* string and the search index always advances past the matched `s1`, replacing `"a"` with `"aa"` terminates instead of looping forever — the classic trap in this exercise.

Errors handled: wrong argument count, unopenable input, uncreatable output, and an empty `s1`. All report to `std::cerr` and exit non-zero.

ex04/Replace.hpp

```
#ifndef REPLACE_HPP
# define REPLACE_HPP

# include <string>

class Replace
{
    private:
        std::string _filename;
        std::string _s1;
        std::string _s2;

        std::string substitute(const std::string& content) const;

    public:
        Replace(std::string filename, std::string s1, std::string s2);
        ~Replace(void);

        bool    run(void) const;
};

#endif
```

ex04/Replace.cpp

```cpp
#include "Replace.hpp"
#include <fstream>
#include <iostream>

Replace::Replace(std::string filename, std::string s1, std::string s2)
    : _filename(filename), _s1(s1), _s2(s2)
{
}

Replace::~Replace(void)
{
}

/*
** std::string::replace is forbidden, so the new content is rebuilt by hand:
** find() locates the next occurrence, substr() copies the text before it, s2
** is appended, and the search resumes AFTER the occurrence. Because the
** result is built in a separate string, an s2 that contains s1 (replacing
** "a" with "aa") cannot loop forever.
*/
std::string Replace::substitute(const std::string& content) const
{
    std::string             result;
    std::string::size_type  start = 0;
    std::string::size_type  pos;

    if (this->_s1.empty())
        return (content);
    while ((pos = content.find(this->_s1, start)) != std::string::npos)
    {
        result += content.substr(start, pos - start);
        result += this->_s2;
        start = pos + this->_s1.length();
    }
    result += content.substr(start);
    return (result);
}

bool    Replace::run(void) const
{
    std::ifstream   infile(this->_filename.c_str());
    std::string     content;
    char            c;

    if (!infile.is_open())
    {
        std::cerr << "Error: cannot open file '" << this->_filename << "'" << std::endl;
        return (false);
    }
    while (infile.get(c))
        content += c;
    infile.close();

    std::string     outname = this->_filename + ".replace";
    std::ofstream   outfile(outname.c_str());

    if (!outfile.is_open())
    {
        std::cerr << "Error: cannot create file '" << outname << "'" << std::endl;
        return (false);
    }
    outfile << this->substitute(content);
    outfile.close();
```

```
    return (true);
}
```

ex04/main.cpp

```
#include "Replace.hpp"
#include <iostream>

int main(int argc, char** argv)
{
    if (argc != 4)
    {
        std::cerr << "Usage: " << argv[0] << " <filename> <s1> <s2>" << std::endl;
        return (1);
    }
    if (std::string(argv[2]).empty())
    {
        std::cerr << "Error: s1 must not be empty" << std::endl;
        return (1);
    }

    Replace sed(argv[1], argv[2], argv[3]);

    if (!sed.run())
        return (1);
    return (0);
}
```

ex04/Makefile

```
NAME            = replace

CXX                     = c++
CXXFLAGS        = -Wall -Wextra -Werror -std=c++98
RM                      = rm -f

SRCS            = main.cpp Replace.cpp
OBJS            = $(SRCS:.cpp=.o)

all: $(NAME)

$(NAME): $(OBJS)
        $(CXX) $(CXXFLAGS) -o $(NAME) $(OBJS)

%.o: %.cpp Replace.hpp
        $(CXX) $(CXXFLAGS) -c $< -o $@

clean:
        $(RM) $(OBJS)

fclean: clean
        $(RM) $(NAME)

re: fclean all

.PHONY: all clean fclean re
```

ACTUAL TEST SESSION

```
$> printf "hello world\nhello again\nno match here\n" > test.txt
$> ./replace test.txt hello HI
$> cat test.txt.replace
HI world
HI again
no match here

$> printf "aaa" > loop.txt          # the s2-contains-s1 trap
$> ./replace loop.txt a aa
$> cat loop.txt.replace   ->  aaaaaa

$> ./replace nosuchfile.txt a b
Error: cannot open file '_nope.txt'
$> ./replace test.txt "" b
Error: s1 must not be empty
$> ./replace test.txt
Usage: ./replace <filename> <s1> <s2>

$> diff <(sed "s/hello/HI/g" test.txt) test.txt.replace   ->  identical
```

EXERCISE 05

Harl 2.0

DIRECTORY	`ex05/`
EXECUTABLE	`harl`
FILES	`Makefile`, `main.cpp`, `Harl.hpp`, `Harl.cpp`

NO IF/ELSE FOREST — THE MECHANISM IS THE GRADE

The four complaint functions share one signature, so a single type describes all of them: `void (Harl::*)(void)`. They are stored in an array, the level name is looked up in a parallel array of strings, and the match is called through its pointer with `(this->*methods[i])()` — the parentheses are mandatory, since `->*` binds looser than the call operator. Adding a fifth level costs two array entries and nothing else.

ex05/Harl.hpp

```
#ifndef HARL_HPP
# define HARL_HPP

# include <string>

class Harl
{
    private:
        void    debug(void);
        void    info(void);
        void    warning(void);
        void    error(void);

    public:
        Harl(void);
        ~Harl(void);

        void    complain(std::string level);
};

#endif
```

ex05/Harl.cpp

```
#include "Harl.hpp"
#include <iostream>

Harl::Harl(void)
{
}

Harl::~Harl(void)
{
}

void    Harl::debug(void)
{
    std::cout << "[ DEBUG ]" << std::endl;
    std::cout << "I love having extra bacon for my 7XL-double-cheese-triple-pickle"
                "-special-ketchup burger. I really do!" << std::endl;
}

void    Harl::info(void)
{
    std::cout << "[ INFO ]" << std::endl;
    std::cout << "I cannot believe adding extra bacon costs more money. You didn't "
                "put enough bacon in my burger! If you did, I wouldn't be asking "
                "for more!" << std::endl;
}

void    Harl::warning(void)
{
    std::cout << "[ WARNING ]" << std::endl;
    std::cout << "I think I deserve to have some extra bacon for free. I've been "
                "coming for years, whereas you started working here just last "
                "month." << std::endl;
}

void    Harl::error(void)
{
    std::cout << "[ ERROR ]" << std::endl;
    std::cout << "This is unacceptable! I want to speak to the manager now." << std::endl;
}

/*
** No if/else forest: the four member functions share one signature, so one
** pointer type describes them all and they can be held in an array. The level
** name is looked up in a parallel array and the matching member function is
** called through its pointer with the ->* operator.
*/
void    Harl::complain(std::string level)
{
    typedef void (Harl::*HarlMethod)(void);

    std::string levels[4] = {"DEBUG", "INFO", "WARNING", "ERROR"};
    HarlMethod  methods[4] = {&Harl::debug, &Harl::info, &Harl::warning, &Harl::error};

    for (int i = 0; i < 4; i++)
    {
        if (levels[i] == level)
        {
            (this->*methods[i])();
            return ;
        }
    }
    std::cout << "[ Probably complaining about insignificant problems ]" << std::endl;
}
```

ex05/main.cpp

```
#include "Harl.hpp"
#include <iostream>

int main(void)
{
    Harl    harl;

    harl.complain("DEBUG");
    harl.complain("INFO");
    harl.complain("WARNING");
    harl.complain("ERROR");
    harl.complain("SOMETHING ELSE");

    return (0);
}
```

ex05/Makefile

```
NAME            = harl

CXX                     = c++
CXXFLAGS        = -Wall -Wextra -Werror -std=c++98
RM                      = rm -f

SRCS            = main.cpp Harl.cpp
OBJS            = $(SRCS:.cpp=.o)

all: $(NAME)

$(NAME): $(OBJS)
        $(CXX) $(CXXFLAGS) -o $(NAME) $(OBJS)

%.o: %.cpp Harl.hpp
        $(CXX) $(CXXFLAGS) -c $< -o $@

clean:
        $(RM) $(OBJS)

fclean: clean
        $(RM) $(NAME)

re: fclean all

.PHONY: all clean fclean re
```

ACTUAL OUTPUT

```
[ DEBUG ]
I love having extra bacon for my 7XL-double-cheese-triple-pickle-special-ketchup burger. I really do!
[ INFO ]
I cannot believe adding extra bacon costs more money. You didn't put enough bacon in my burger! If you did,
I wouldn't be asking for more!
[ WARNING ]
I think I deserve to have some extra bacon for free. I've been coming for years, whereas you started
working here just last month.
[ ERROR ]
This is unacceptable! I want to speak to the manager now.
[ Probably complaining about insignificant problems ]
```

EXERCISE 06 — OPTIONAL

Harl filter

DIRECTORY	`ex06/`
EXECUTABLE	`harlFilter` (name is mandatory)
FILES	`Makefile`, `main.cpp`, `Harl.hpp`, `Harl.cpp`

FALLTHROUGH IS THE FEATURE, NOT A BUG

`switch` only accepts an integral value, so `getLevel()` converts the level name into an index first — `-1` for anything unrecognised, which lands in `default`. The cases are ordered least to most severe and deliberately omit `break`: matching `WARNING` falls straight through into `ERROR`, which is exactly "this level and every level above it". Only the last case breaks.

ex06/Harl.hpp

```
#ifndef HARL_HPP
# define HARL_HPP

# include <string>

class Harl
{
    private:
        void    debug(void);
        void    info(void);
        void    warning(void);
        void    error(void);

        int     getLevel(std::string level) const;

    public:
        Harl(void);
        ~Harl(void);

        void    complain(std::string level);
};

#endif
```

ex06/Harl.cpp

```
#include "Harl.hpp"
#include <iostream>

Harl::Harl(void)
{
}

Harl::~Harl(void)
{
}

void    Harl::debug(void)
{
    std::cout << "[ DEBUG ]" << std::endl;
    std::cout << "I love having extra bacon for my 7XL-double-cheese-triple-pickle"
                "-special-ketchup burger. I really do!" << std::endl;
}

void    Harl::info(void)
{
    std::cout << "[ INFO ]" << std::endl;
    std::cout << "I cannot believe adding extra bacon costs more money. You didn't "
                "put enough bacon in my burger! If you did, I wouldn't be asking "
                "for more!" << std::endl;
}

void    Harl::warning(void)
{
    std::cout << "[ WARNING ]" << std::endl;
    std::cout << "I think I deserve to have some extra bacon for free. I've been "
                "coming for years, whereas you started working here just last "
                "month." << std::endl;
}

void    Harl::error(void)
{
    std::cout << "[ ERROR ]" << std::endl;
    std::cout << "This is unacceptable! I want to speak to the manager now." << std::endl;
}

/*
** switch only accepts an integral value, never a std::string, so the level
** name is turned into an index first. -1 means "unknown" and lands in default.
*/
int     Harl::getLevel(std::string level) const
{
    std::string levels[4] = {"DEBUG", "INFO", "WARNING", "ERROR"};

    for (int i = 0; i < 4; i++)
    {
        if (levels[i] == level)
            return (i);
    }
    return (-1);
}

/*
** The missing break statements are deliberate. Matching a case falls through
** into every case below it, so the requested level and all the more severe
** ones are printed. Only the last case breaks.
*/
void    Harl::complain(std::string level)
{
    switch (this->getLevel(level))
```

```
    {
        case 0:
            this->debug();
            /* fall through */
        case 1:
            this->info();
            /* fall through */
        case 2:
            this->warning();
            /* fall through */
        case 3:
            this->error();
            break ;
        default:
            std::cout << "[ Probably complaining about insignificant problems ]"
                      << std::endl;
            break ;
    }
}
```

ex06/main.cpp

```
#include "Harl.hpp"
#include <iostream>

int main(int argc, char** argv)
{
    if (argc != 2)
    {
        std::cerr << "Usage: " << argv[0] << " <DEBUG|INFO|WARNING|ERROR>" << std::endl;
        return (1);
    }

    Harl    harl;

    harl.complain(argv[1]);
    return (0);
}
```

ex06/Makefile

```
NAME            = harlFilter

CXX                     = c++
CXXFLAGS        = -Wall -Wextra -Werror -std=c++98
RM                      = rm -f

SRCS            = main.cpp Harl.cpp
OBJS            = $(SRCS:.cpp=.o)

all: $(NAME)

$(NAME): $(OBJS)
        $(CXX) $(CXXFLAGS) -o $(NAME) $(OBJS)

%.o: %.cpp Harl.hpp
        $(CXX) $(CXXFLAGS) -c $< -o $@

clean:
        $(RM) $(OBJS)

fclean: clean
        $(RM) $(NAME)

re: fclean all

.PHONY: all clean fclean re
```

ACTUAL OUTPUT — MATCHES THE SUBJECT EXACTLY

```
$> ./harlFilter "DEBUG"
[ DEBUG ]
I love having extra bacon for my 7XL-double-cheese-triple-pickle-special-ketchup burger. I really do!
[ INFO ]
I cannot believe adding extra bacon costs more money. You didn't put enough bacon in my burger! If you did,
I wouldn't be asking for more!
[ WARNING ]
I think I deserve to have some extra bacon for free. I've been coming for years, whereas you started
working here just last month.
[ ERROR ]
This is unacceptable! I want to speak to the manager now.

$> ./harlFilter "WARNING"
[ WARNING ]
I think I deserve to have some extra bacon for free. I've been coming for years, whereas you started
working here just last month.
[ ERROR ]
This is unacceptable! I want to speak to the manager now.

$> ./harlFilter "ERROR"
[ ERROR ]
This is unacceptable! I want to speak to the manager now.

$> ./harlFilter "I am not sure how tired I am today..."
[ Probably complaining about insignificant problems ]

$> ./harlFilter
Usage: ./harlFilter <DEBUG|INFO|WARNING|ERROR>
```

CLOSING

Defending this at evaluation

The subject states that a brief live modification may be requested during the defense — a small behaviour change, a few lines to rewrite, an easy feature to add. Working code is not sufficient on its own; you have to be able to explain and modify it. These are the questions this module is built around, one per exercise.

EX	WHAT YOU WILL BE ASKED
ex00	Why heap in `newZombie` and stack in `randomChump`? What happens at the closing brace of `randomChump`? Who deletes what `newZombie` returns?
ex01	Why `delete[]` and not `delete`? How does the runtime know how many destructors to run? Why can't the name be passed to `new Zombie[N]`?
ex02	Why are all three addresses the same? Name three things a pointer can do that a reference cannot. Does a reference occupy memory of its own?
ex03	Why a reference in HumanA and a pointer in HumanB — could you swap them? Why does the second `attack()` show the new weapon? What breaks if `getType()` returns by value?
ex04	What does `find` return when there is no match, and what type is it? Trace the loop with `s1 = "a"`, `s2 = "aa"`. What happens on an empty or unreadable file?
ex05	Why is a member function pointer not a normal function pointer? What does `this->*` do? How many lines does adding a fifth level cost?
ex06	Why can't `switch` take a `std::string`? Explain fallthrough — why a bug usually, and correct here? What happens with no argument?

Rebuilding everything from scratch

```
for d in ex00 ex01 ex02 ex03 ex04 ex05 ex06; do (cd $d && make re); done
```

Each directory is self-contained: its own `Makefile`, its own sources, no shared files between exercises. `ex06` is an extended copy of `ex05`'s `Harl`, as the identical file names in the subject suggest.